What must be ready by deadline.

1. The actual system you must have ready

Your group should have a working **microservices-based healthcare platform** with an **asynchronous web client**. The assignment specifically expects RESTful services, microservices architecture, Docker, Kubernetes, role-based authentication, and a JavaScript-based async client.

So by deadline, your project should be ready with these main parts:

Core functional parts

- Patient side
  - register/login
  - manage profile
  - browse/search doctors
  - book appointment
  - modify/cancel booking
  - upload medical reports
  - view prescriptions
  - attend video consultation
- Doctor side
  - doctor profile management
  - set availability
  - accept/reject appointments
  - conduct telemedicine/video session
  - issue digital prescriptions
  - view patient uploaded reports
- Admin side
  - manage users
  - verify doctor registrations
  - oversee platform operations
  - handle/monitor financial side
- Appointment service
  - search doctor by specialty
  - booking flow
  - cancel/update booking
  - real-time appointment status tracking
- Telemedicine service
  - third-party video integration like Jitsi / Agora / Twilio
- Payment service
  - payment gateway integration, at least sandbox/demo flow
- Notification service
  - SMS/email confirmation for booking and/or consultation completion
- AI symptom checker
  - this is **optional enhancement**, not compulsory.

2. Technical things that must be ready

Even if your app is running from VS Code during development, that alone is not enough. The spec wants these:

Must-have technical expectations

- **RESTful backend services**
- **Microservices architecture**
- **Docker**
- **Kubernetes**
- **Asynchronous web client** using React/Angular/jQuery+AJAX
- **Authentication/security**
- **3 roles**: Patient, Doctor, Admin

So for your current stack, that means:

- VS Code = okay as your editor
- MongoDB Atlas = okay as your DB choice
- Docker = must be used
- Kubernetes = must also be shown/used
- React frontend = okay for async web client
- Node/Express microservices = okay if that is your backend style

3. What you must submit as files

The submission is not just source code. You need these files ready:

Required submission files

1. **submission.txt**
    - must contain your **GitHub repository link**
    - must also contain your **YouTube presentation/demo link**
2. **readme.txt**
    - must contain the steps to deploy/run the project
3. **members.txt**
    - names
    - registration numbers
    - IDs of all group members
4. **report.pdf**
    - compulsory report in PDF format
5. Everything must be uploaded in **one ZIP file**
    - ZIP name format: **Group ID_DS-Assignment.zip**

4. What the report must contain

Your **report.pdf** must include these sections:

- high-level architectural diagram showing services and how they connect
- list of **service interfaces** exposed by each service

- brief explanation of each workflow in the system
- authentication/security mechanisms used
- individual contribution of each group member

Also important:

- you may use code snippets in the report
- the report must have an **appendix with all code written by you**
- do **not** paste screenshots of code in appendix
- code must be copied as **text**
- if you use screenshots instead, only minimum marks may be given

And also:

- report goes to **Turnitin**
- if similarity is **above 20%**, marks will be penalized

5. What the video must be like

You need a **YouTube presentation/demo link** inside `submission.txt`.
The video rules are:

- each member can explain for **maximum 3 minutes**
- total video length should **not exceed 12 minutes**

So your team should prepare:

- short architecture intro
- each person's contribution demo
- final system flow demo

6. What should realistically be ready by the deadline for your project

Since you are already building the healthcare system, by deadline you should make sure these are all ready:

Minimum safe-ready state

- frontend works
- backend microservices run
- database works with MongoDB Atlas
- Dockerfiles ready for services
- Kubernetes manifests ready and tested
- login/auth works
- Patient, Doctor, Admin roles work
- appointment booking flow works
- doctor availability works
- telemedicine demo works, even if basic
- notifications work, at least demo mode

- payment works, at least sandbox/demo mode
- GitHub repo cleaned and organized
- report complete
- YouTube demo recorded
- all required `.txt` files prepared
- final ZIP prepared

7. Very important thing for your specific case

Because you said you are using **VS Code, Docker, and MongoDB Atlas**, here is the truth:

That is okay, but not enough by itself

You still need to be able to show:

- that the app is built as **microservices**
- that **Docker** is used
- that **Kubernetes** is used or demonstrated
- that the client is **asynchronous**
- that roles and security are implemented

So if your team has only coded services locally in VS Code and used Atlas, but has **not done Kubernetes yet**, then that part is still pending according to the assignment.

8. Final one-shot checklist for submission day

By the deadline, keep these ready:

- working healthcare microservices project
- async frontend
- Patient / Doctor / Admin roles
- authentication/security
- appointment flow
- doctor management
- patient management
- telemedicine integration
- payment integration
- notification integration
- Docker setup
- Kubernetes setup
- GitHub repo
- YouTube demo video
- `submission.txt`
- `readme.txt`
- `members.txt`
- `report.pdf`
- appendix with code as text
- one final ZIP named properly

9. Best way to think about it

Think of the deadline as needing **3 things fully ready**:

A. Working project
Your healthcare platform should run and demonstrate the main flows.

B. Proof/documentation
Report, deployment steps, architecture, interfaces, workflows, security, contributions.

C. Submission packaging
All required files + one ZIP + repo link + video link.

Report content

Recommended report structure

1. Cover page
2. Introduction
3. System overview
4. High-level architectural diagram
5. Service interfaces of each microservice
6. Workflows of the system
7. Authentication and security mechanisms
8. Docker and Kubernetes deployment overview
9. Individual contributions
10. Conclusion
11. Appendix – code written by the team

That structure fits your assignment nicely because it directly covers all required report sections.

2. "High-level architectural diagram showing services and how they connect" — what this means

This does **not** mean drawing every file, every folder, or every line of code.

It means showing the **big picture** of your system.

For your healthcare microservices project, this diagram should show things like:

- React frontend
- API gateway or frontend calling services directly
- Auth service
- Patient service
- Doctor service
- Appointment service
- Telemedicine service
- Payment service
- Notification service
- database(s)
- external APIs like:
    - video API
    - payment gateway
    - email/SMS provider
- Docker/Kubernetes environment

What the lecturer wants to see from this diagram

They want to quickly understand:

- what services exist
- which service talks to which service

- where users enter the system
- where third-party integrations happen
- how the overall platform is organized

---

3. How to do that architectural diagram step by step

Use something simple like:

- **draw.io**
- **Lucidchart**
- **Canva**
- even PowerPoint shapes if needed

Step-by-step

Step 1: List all major components

Write them on paper first.

Example for your project:

- Patient user
- Doctor user
- Admin user
- React frontend
- Auth service
- Patient service
- Doctor service
- Appointment service
- Telemedicine service
- Payment service
- Notification service
- MongoDB Atlas
- External video API
- External payment API
- Email/SMS provider
- Kubernetes/Docker environment

Step 2: Put users at the top or left

Draw:

- Patient
- Doctor
- Admin

Then connect them to:

- Web Client / React Frontend

Step 3: Put frontend in the middle

Draw one box:

- **Asynchronous Web Client (React)**

Step 4: Add backend services

Below the frontend, draw separate boxes:

- Auth Service
- Patient Service
- Doctor Service
- Appointment Service
- Telemedicine Service
- Payment Service
- Notification Service

Step 5: Add database

Draw MongoDB Atlas below or to the side.

Then connect the services that use it.

Step 6: Add external systems

Draw boxes for:

- Jitsi / Agora / Twilio
- PayHere / Stripe / PayPal
- Email/SMS service

Then connect:

- Telemedicine Service → Video API
- Payment Service → Payment Gateway
- Notification Service → Email/SMS provider

Step 7: Mention deployment

Add a light outer box around backend services saying:

- **Dockerized Microservices deployed using Kubernetes**

That shows you understood the architecture requirement.

Step 8: Keep it high-level

Do **not** add:

- controller class names
- every endpoint
- every folder
- every schema field

This section is only for the architecture overview.

---

4. What to write below that diagram in the report

After inserting the diagram, add a short explanation like this:

Example writing

"This healthcare platform is designed using a microservices architecture. The React-based asynchronous web client is used by patients, doctors, and administrators to access the system. Requests are handled by independent backend services such as authentication, patient management, doctor management, appointment scheduling, telemedicine, payment, and notification services. These services interact with MongoDB Atlas for data storage and use external third-party APIs for video consultation, payment processing, and email/SMS notifications. The services are containerized using Docker and intended to be orchestrated using Kubernetes for scalable deployment."

That is the kind of explanation they expect.

---

5. "List of service interfaces exposed by each service" — what this means

This means **API endpoints**, not screens.

The assignment even says **NOT user interfaces, but service interfaces**.

So here you must list things like:

Example

Auth Service

- `POST /api/auth/register` – register user
- `POST /api/auth/login` – login user
- `GET /api/auth/me` – get logged-in user

Patient Service

- `GET /api/patients/me`
- `PUT /api/patients/me`
- `POST /api/patients/reports`

Doctor Service

- `GET /api/doctors`
- `GET /api/doctors/:id`
- `PUT /api/doctors/me`
- `POST /api/doctors/availability`

Appointment Service

- `POST /api/appointments`
- `GET /api/appointments/my`
- `PATCH /api/appointments/:id/cancel`

Telemedicine Service

- `POST /api/telemedicine/session`
- `GET /api/telemedicine/session/:appointmentId`

Payment Service

- `POST /api/payments/create`
- `POST /api/payments/confirm`

Notification Service

- `POST /api/notifications/email`
- `POST /api/notifications/sms`

How to present this

Use a table:

Service	Method	Endpoint	Purpose	Access Role

That is the cleanest way.

---

6. “Brief explanation of each workflow” — what this means

A workflow means a **real system process from start to finish**.

For your healthcare system, good workflows are:

- User registration and login
- Doctor verification by admin
- Booking an appointment
- Cancelling/modifying an appointment
- Doctor accepting/rejecting appointment
- Starting a telemedicine session
- Uploading medical reports
- Issuing prescriptions
- Making payment
- Sending notifications

How to write a workflow

For each workflow, write:

1. name of workflow
2. short goal
3. step-by-step flow
4. which services are involved

Example: Appointment Booking Workflow

Goal: Allow a patient to reserve a consultation slot with a doctor.

Steps:

1. Patient logs into the system.
2. Patient searches doctors by specialty.
3. Frontend requests doctor data from Doctor Service.
4. Patient selects a doctor and available time slot.
5. Frontend sends booking request to Appointment Service.
6. Appointment Service validates patient, doctor, and slot availability.
7. Booking is saved in the database.
8. Payment Service may be triggered if prepayment is required.
9. Notification Service sends confirmation to patient and doctor.
10. Updated appointment status is shown in the frontend.

Optional extra

You can draw a simple sequence diagram for 2–3 important workflows.
That would make the report look much better.

---

7. “Authentication/security mechanisms adopted” — what to put here

This section should explain how you protected the system.

For your kind of project, usually include things like:

- JWT-based authentication
- password hashing using bcrypt
- role-based authorization
- protected API routes
- input validation
- CORS configuration
- environment variables for secrets
- HTTPS concept for production
- token verification middleware
- restricting endpoints by role:
    - Patient
    - Doctor
    - Admin

Example writing

“The system uses JWT-based authentication to verify users after login. Passwords are securely hashed before storage. Protected routes are enforced using authentication middleware, and role-based authorization is used to restrict access to specific endpoints. For example, only administrators can verify doctors, while patients can create appointments and doctors can manage their availability and prescriptions. Sensitive configuration values such as JWT secrets and API keys are stored using environment variables.”

That is the kind of content you need.

---

8. “Individual contributions of each group member” — what to do

This part is simple.

Make a table like this:

Member	Registration No	Contribution

Member 1	ITxxxxxxx	Auth service, JWT, login/register APIs
Member 2	ITxxxxxxx	Patient module, report upload
Member 3	ITxxxxxxx	Appointment and doctor services
Member 4	ITxxxxxxx	Telemedicine, notifications, deployment

Be honest. Keep it short and clear.

---

9. “Appendix with all code written by you” — how to do it properly

The assignment says:

- appendix must contain **all code written by you**
- **excluding auto-generated code**
- code must be pasted as **text**
- not screenshots

What should go into appendix

Include only code your team actually wrote or meaningfully modified, such as:

- controllers
- routes
- middleware
- models/schemas
- service logic
- utility functions
- frontend pages/components you wrote
- Dockerfiles
- docker-compose
- Kubernetes YAML files
- important config files you manually wrote

What should usually NOT go into appendix

Do not include huge auto-generated content like:

- `node_modules`
- package lock giant content
- Vite default boilerplate if unchanged
- React scaffold code if untouched
- generated build files
- copied library code
- external SDK code

Best way to organize appendix

Do this:

Appendix A – Backend Code

- Auth Service
- Patient Service
- Doctor Service
- Appointment Service
- Telemedicine Service
- Payment Service
- Notification Service

Appendix B – Frontend Code

- Login page
- Dashboard pages
- Appointment booking component
- Telemedicine UI

Appendix C – Deployment Code

- Dockerfiles
- docker-compose.yml
- Kubernetes manifests

---

10. Your Turnitin concern — very important

You asked a very real question:

> "We copied the code you gave straight into VS Code. Then if we paste that code in the appendix, won't Turnitin similarity go above 20%? How do we solve that?"

I need to answer this honestly.

The honest truth

If you submit large amounts of copied material that are not truly your own wording or not genuinely adapted by your team, **there is a real plagiarism/similarity risk**.

I cannot help you “trick” Turnitin or hide copied work.

But I **can** tell you the proper and safe way to reduce legitimate similarity risk.

---

11. Safe and proper ways to handle the Turnitin issue

A. Do not submit raw untouched copied code blindly

Since you said the code was largely copy-pasted, do not just leave it exactly as-is without understanding it.

What you should do instead:

- review every file
- understand every endpoint
- rename things only where it makes design sense
- remove unused code
- rewrite comments in your own words
- adjust structure to match your actual project
- test each part
- fix bugs yourselves
- make sure the final code reflects your actual implementation

That makes the code part of your real project work, not just blind copy-paste.

B. Write all report explanations in your own words

This is very important.

Turnitin does not only look at code-like material; it also checks text similarity.

So:

- do not copy explanation paragraphs from anywhere
- write architecture explanation in your own simple English
- write workflows in your own words
- write security explanation in your own words
- write contribution section in your own words

This alone helps a lot.

C. Include only code that the assignment asks for

The assignment says appendix should contain all code written by you, excluding auto-generated code.

So do not dump unnecessary files just to make appendix huge.

Only include relevant handwritten code.

D. Separate auto-generated and handwritten code properly

Since the sheet explicitly says exclude auto-generated code, that helps you.

So in appendix, include:

- files you actually worked on

Do not include:

- scaffold-generated junk
- dependency code
- build output
- large template code not authored by your team

E. Add a short declaration of tool-assisted development if your institution allows it

Some lecturers are okay if you say the team used tools for assistance but reviewed, integrated, tested, and customized the implementation.

Something like:
"Code development was supported by standard development tools and then reviewed, integrated, tested, and adapted by the team to fit the system requirements."

Only do this if it fits your university norms.

F. Ask whether appendix can be treated separately for similarity

This is important.

Sometimes similarity tools count appendices heavily. Sometimes instructors manually focus on the main body.
You should ask your lecturer or module coordinator:

"Since the appendix must contain source code as text, will the appendix be considered separately when checking similarity?"

That is a fair and honest academic question.

Do not assume.

---

12. What you should NOT do

Do not try to:

- artificially replace words with weird synonyms
- deliberately obfuscate code
- use random variable renaming only to “beat Turnitin”
- paraphrase dishonestly without understanding
- submit code you cannot explain in viva

Because even if similarity is low, during viva they can immediately tell whether you understand it.

And for this module, viva/report/demo all matter together.

---

13. Best practical solution for your team right now

Since your project is still being built, do this from today onward:

For code

- keep only the files you actually use
- clean the codebase
- remove dead code
- ensure each service really works
- make small real edits where needed
- understand every route, model, and middleware

For report

- write the report fully in your own words
- use your actual project structure
- include only meaningful code snippets in main body
- put full handwritten code in appendix
- exclude auto-generated code

For viva safety

Every team member should be able to explain:

- their service
- their endpoints
- DB fields
- why Docker is used
- why Kubernetes is used
- how auth works
- how notifications/payments/video work

If you cannot explain the code, that is more dangerous than Turnitin.

---

14. My direct answer to your question

Yes:

All those report contents must be there.

For the architectural diagram:

You make one big-picture diagram showing users, frontend, microservices, DB, and external APIs, then explain it in 1 short paragraph.

For the service interfaces:

List API endpoints service by service in a table.

For workflows:

Explain the major system processes step by step.

For security:

Explain auth, JWT, roles, route protection, password hashing, and related mechanisms.

For appendix:

Paste only your handwritten code as text, not screenshots, and exclude auto-generated code.

For Turnitin:

You should not try to “bypass” it.
The proper way is to make the implementation genuinely yours through review, integration, testing, cleanup, adaptation, and original report writing. Also ask the lecturer whether the appendix code is evaluated separately for similarity.

---

15. Best next move

The most useful next step is this:

I can build you the **full report outline section by section** for your healthcare project in a very human student style, starting with:

- **1. Introduction**
- **2. System Overview**
- **3. High-Level Architecture Diagram section**
- **4. Service Interfaces section**

and I'll write it exactly in a report-ready way.